

CSV Import QA Assessment

1. Requirements Review

Assumptions

- The current system date is used to determine whether an order is older than 365 days.
 - An order is considered valid only if:
 - `order_id` is unique;
 - `amount` is greater than zero;
 - `currency` is one of the supported values (USD, EUR, GBP).
 - If duplicate orders are present in the source file, only one record should be imported.
 - Customer IDs are compared in a case-insensitive manner.
 - Invalid rows must not stop the import process.
 - The import summary is generated after all rows have been processed.
-

Ambiguities

1. Is an order exactly **365 days old** considered valid?
 2. Are future order dates allowed?
 3. Should `customer_id` values be normalized (e.g., converted to uppercase) after import?
 4. If duplicate orders exist, should the first or the last occurrence be imported?
 5. How should rows with empty mandatory fields be handled?
 6. Are there any requirements regarding decimal precision or rounding of the `amount` field?
-

Questions for the Product Team

1. Should the rejection reason for `amount = 0` be explicitly reported as `non_positive_amount`?
 2. Are future order dates expected to be accepted?
 3. Should imported customer IDs preserve the original case or be normalized?
 4. Which duplicate record should be kept?
 5. Is the CSV format guaranteed (date format, encoding, delimiter)?
 6. Should leading and trailing spaces be trimmed automatically?
-

Risks

- Incorrect financial totals in the import summary.
- Duplicate orders imported into production.

- Invalid orders accepted.
 - Valid orders rejected incorrectly.
 - Partial imports without proper reporting.
 - Customer matching issues caused by inconsistent casing.
 - Reconciliation problems due to incorrect reporting.
-

2. Test Strategy

Highest-Risk Scenarios

- Duplicate `order_id`
 - Unsupported currency
 - Negative amount
 - Zero amount
 - Orders older than 365 days
 - Import summary calculation
 - Import continuation after invalid records
 - Large CSV imports
-

Boundary Cases

- `amount = 0`
 - `amount = 0.01`
 - Maximum supported amount
 - Order exactly 365 days old
 - Order 366 days old
 - Empty mandatory fields
 - Empty `customer_id`
 - Lowercase vs uppercase customer IDs
 - Duplicate records located next to each other
 - Duplicate records located far apart
-

Production Risks

- Incorrect imported amount displayed to users.
- Duplicate financial transactions.

- Missing imported records.
 - Import process terminating after the first invalid row.
 - Incorrect import statistics.
 - Data inconsistency between source file and database.
-

3. Imported Data Validation

Files Compared

- orders.csv
 - import_log.log
 - imported_orders.csv
-

Validation Results

Import Statistics

Metric	Expected	Actual	Status
Total rows	15	15	Passed
Imported rows	11	11	Passed
Rejected rows	5	4	Failed

Duplicate Handling

The source file contains duplicate order **1008**.

Only one record was imported.

Status: **Passed**

Unsupported Currency

Orders with unsupported currency were rejected.

Status: **Passed**

Negative Amount

Orders with negative amounts were rejected.

Status: **Passed**

Orders Older Than 365 Days

Old orders were rejected correctly.

Status: **Passed**

Zero Amount Validation

Order **1007** has amount = 0.

According to the requirement:

Amount should be positive.

The record was imported.

Status: **Failed**

Imported Amount Validation

Import summary reports:

1,003,574.99

Actual total amount of imported records:

1,002,574.99

Difference:

1,000.00

Status: **Failed**

Customer ID Handling

The requirements state that customer IDs are case-insensitive.

Imported data contains customer IDs with different casing.

Although this does not necessarily violate the requirements, the expected behavior should be clarified.

Status: Requires clarification

Validation Summary

Validation	Result
Imported row count	Passed
Rejected row count	Failed
Duplicate handling	Passed
Unsupported currency validation	Passed
Negative amount validation	Passed
Old order validation	Passed
Zero amount validation	Failed
Imported amount calculation	Failed
Customer ID normalization	Requires clarification

4. Automation

A Python validation script was created to automatically verify the consistency of the import process.

The automation performs the following checks:

- compares the source CSV with imported data;
- validates duplicate handling;
- verifies imported and rejected row counts;
- validates the total imported amount;

- checks supported currencies;
- verifies that imported amounts are positive;
- generates a human-readable validation report.

This automation can be integrated into a CI/CD pipeline to validate every CSV import automatically.

5. Bug Reports

Bug #1

Title

Import summary displays an incorrect total imported amount

Severity

High

Preconditions

- Import the provided `orders.csv` file.

Steps to Reproduce

1. Import `orders.csv`.
2. Wait until the import completes.
3. Compare the total imported amount displayed in the import summary with the sum of all imported records.

Expected Result

The total imported amount displayed in the import summary matches the sum of all successfully imported records.

Actual Result

Import summary amount:

1,003,574.99

Actual imported amount:

1,002,574.99

Difference:

1,000.00

Bug #2

Title

Order with zero amount is imported

Severity

High

Preconditions

The source CSV contains the following record:

- `order_id = 1007`
- `amount = 0`

Steps to Reproduce

1. Import `orders.csv`.
2. Wait until the import completes.
3. Review the imported records.

Expected Result

The order is rejected because the amount must be positive.

Actual Result

The order is imported successfully despite having an amount of 0.

Bug #3

Title

Import summary displays an incorrect rejected rows count

Severity

Medium

Preconditions

The source CSV contains five records that should be rejected according to the import validation rules:

- 1004 — order older than 365 days

- 1005 — unsupported currency (CAD)
- 1006 — negative amount
- 1007 — zero amount
- 1008 — duplicate `order_id`

Steps to Reproduce

1. Import `orders.csv`.
2. Wait until the import completes.
3. Compare the rejected rows count displayed in the import summary with the expected number of rejected records.

Expected Result

The import summary displays:

Rejected rows: 5

Actual Result

The import summary displays:

Rejected rows: 4

The invalid record with `order_id = 1007` is imported instead of being rejected, resulting in an incorrect rejected rows count.

Potential Issue / Requirement Gap

Title

Future-dated orders are accepted without explicit validation

Severity

Medium

Preconditions

The source CSV contains an order with:

- `order_id = 1011`
- `order_date = 2030-01-01`

Steps to Reproduce

1. Import `orders.csv`.
2. Wait until the import completes.
3. Review the imported records.

Expected Result

The system should either:

- reject future-dated orders; **or**
- explicitly document that future dates are supported.

Actual Result

The order is imported successfully even though its date is in the future.

Notes

The current requirements specify that orders older than **365 days** must be rejected, but they do not define the expected behavior for **future-dated orders**. This should be clarified with the Product Team before treating it as a confirmed defect.

Automation Test Execution

Objective

Verify consistency between:

- source data (`orders.csv`);
- imported data (`imported_orders.csv`);
- import summary (`import_log.log`).

Test Environment

Property	Value
Language	Python 3
Framework	Pytest
Validation Module	<code>order_import_validator.py</code>
Test Suite	<code>test_order_import_validation.py</code>

Execution Summary

Metric	Result
Total Tests	10
Passed	7
Failed	3

Test Results

Test	Status	Notes
test_total_rows_count_matches_import_log	Passed	Total rows in the source file match the import log.
test_imported_rows_count_matches_import_log	Passed	Imported rows count matches the import log.
test_rejected_rows_count_matches_expected_rejections	Failed	The import log reports 4 rejected records, while validation identifies 5 records that should be rejected.
test_imported_data_does_not_contain_duplicate_order_ids	Passed	No duplicate order_id values were found in the imported data.
test_all_expected_valid_orders_are_imported	Passed	All valid source records were successfully imported.
test_invalid_orders_are_not_imported	Failed	Invalid order 1007 (amount = 0) was imported instead of being rejected.
test_imported_amount_matches_expected_amount	Passed	Imported records total matches the expected amount calculated from valid source records.
test_import_log_total_amount_matches_imported_data	Failed	The total imported amount in the log

Test	Status	Notes
		does not match the imported data.
test_future_dated_imported_orders_are_reported_as_requirement_gap	Passed	Future-dated orders are detected and reported as a requirement gap.
test_customer_id_case_conflicts_are_reported_as_requirement_gap	Passed	Customer ID casing inconsistencies are detected and reported as a requirement gap.

Summary

The automated validation identified **three confirmed defects**, which correspond to the reported bugs:

- **Bug #1** — Incorrect total imported amount in the import summary.
- **Bug #2** — Order with zero amount is imported.
- **Bug #3** — Incorrect rejected rows count in the import summary.

Additionally, the automation highlights **future-dated orders** and **customer ID casing inconsistencies** as requirement gaps that should be clarified with the Product Team before being considered defects.

Conclusion

The import functionality correctly handles:

- duplicate orders;
- unsupported currencies;
- negative amounts;
- orders older than 365 days;
- partial imports without stopping the process.

However, the validation identified several important issues:

1. The total imported amount reported in the import summary is incorrect.
2. An order with an amount of 0 is imported despite the requirement that the amount must be positive.
3. The rejected rows count in the import summary is incorrect.
4. The expected behavior for customer ID normalization is not clearly specified.

Requirement gaps:

1. Future-dated orders are accepted, but the expected behavior is not defined in the requirements

Overall, the core import functionality works as expected for most validation rules, but the identified defects affect data integrity and financial reporting. These issues should be resolved before the feature is considered production-ready.